



DESARROLLO BIG DATA CON PYTHON

TEMA 4. Spark SQL y
Procesamiento de Datos



Contenidos Tema 4

- / 1. Fundamentos de Spark SQL**
- / 2. Operaciones con DataFrames: selecciones, filtrado, agrupaciones y joins**
- / 3. Uso de funciones integradas y procesamiento**
- / 4. LAB: Limpieza y análisis exploratorio (EDA) con consultas SQL nativas**

¿Qué es Spark SQL?

- Un módulo central de Spark, especializado en el procesamiento de datos estructurados.
- El cerebro detrás del motor de Spark SQL tiene un nombre → **Catalyst Optimizer**: optimizador automático que reescribe tu código antes de su ejecución para maximizar el rendimiento.



- Gracias al optimizador **Catalyst**, el código se ejecuta con la máxima velocidad y eficiencia posibles, independientemente de cómo se hayan escrito aspectos como las consultas SQL.

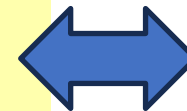
1. Fundamentos de Spark SQL

Spark SQL: Dualidad de Sintaxis

- **PySpark DataFrame API:** basado en ideas de Pandas y en orientación a objetos de Python. Más cómodo para desarrolladores y para modelar cadenas de transformaciones complejas.
- **SQL Nativo:** sintaxis clásica de consulta a bases de datos, más cómodo para analistas o para realizar consultas exploratorias rápidas y de menor complejidad.
- Dos enfoques **100% interoperables:** puedes empezar con la API de DataFrame para limpiar datos, y después agregarlos usando SQL si lo prefieres.

```
# --- Enfoque 1: DataFrame API (Python, similar a
Pandas) ---
df_resultado = df.filter(df.edad >= 18) \
    .groupBy("ciudad") \
    .count()

df_resultado.show()
```



```
# --- Enfoque 2: SQL Nativo ---
df_resultado = spark.sql("""
    SELECT ciudad, COUNT(*) as count
    FROM usuarios
    WHERE edad >= 18
    GROUP BY ciudad
""")

df_resultado.show()
```

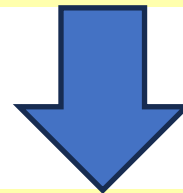
```
+-----+-----+
| ciudad|count|
+-----+-----+
| London| 2|
|New York| 3|
+-----+-----+
```

1. Fundamentos de Spark SQL

Spark SQL: Vistas Temporales

- Spark SQL no puede leer un DataFrame de Python directamente usando código SQL.
- **Solución:** creación de una **vista temporal**, registrando un DataFrame en el catálogo de metadatos de Spark → esto permite que los datos sean "visibles" a un comando SQL como si de una tabla se tratara.

```
# Registramos el DataFrame como una vista temporal para poder utilizar Spark SQL
df.createOrReplaceTempView("usuarios")
```



```
# --- Enfoque 2: SQL Nativo ---
df_resultado = spark.sql("""
    SELECT ciudad, COUNT(*) as count
    FROM usuarios
    WHERE edad >= 18
    GROUP BY ciudad
    """)
```

SELECCIÓN Y DERIVACIÓN DE DATOS

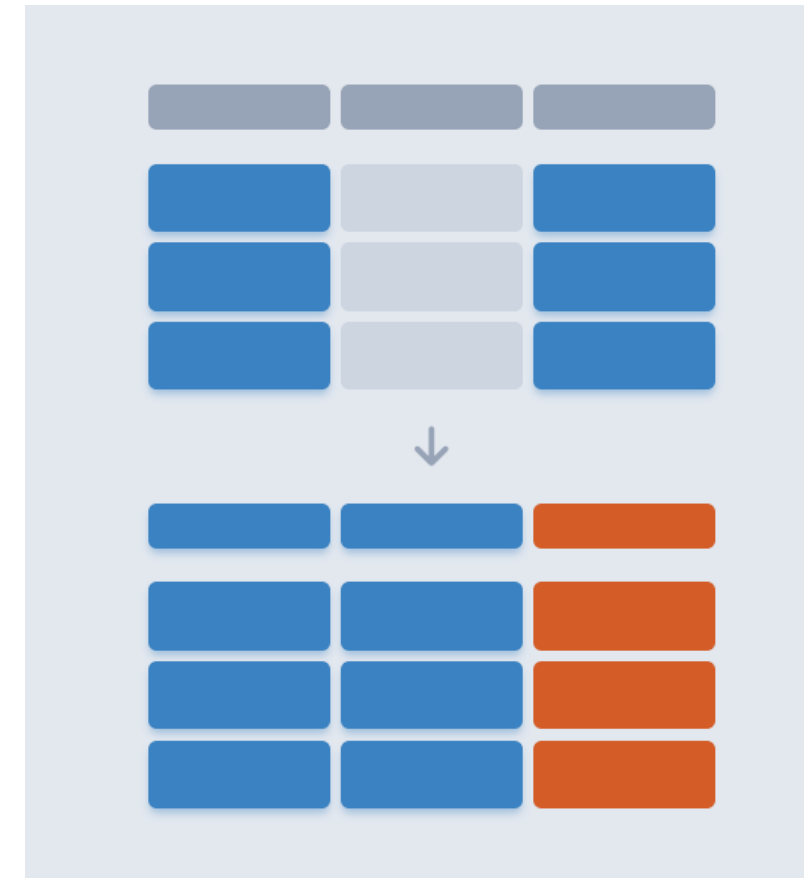
Extraer columnas específicas y crear nuevas variables a partir de los datos existentes.

Seleccionar dos columnas por nombre y crear una nueva columna resultado de aplicar IVA del 21% a los valores de la columna "ventas":

```
from pyspark.sql.functions import col, round

# 1. DataFrame API
df.select(
    col("id_usuario"), col("ventas")
).withColumn("con_iva", round(col("ventas") * 1.21, 2)).show()
```

```
# 2. Spark SQL
spark.sql("""
    SELECT id_usuario, ventas,
    (ventas * 1.21) AS con_iva
    FROM tabla_ventas
""").show()
```



2. Operaciones con DataFrames en PySpark

FILTRADO DE DATOS

Extraer únicamente las filas cuyos valores cumplan alguna condición lógica especificada.

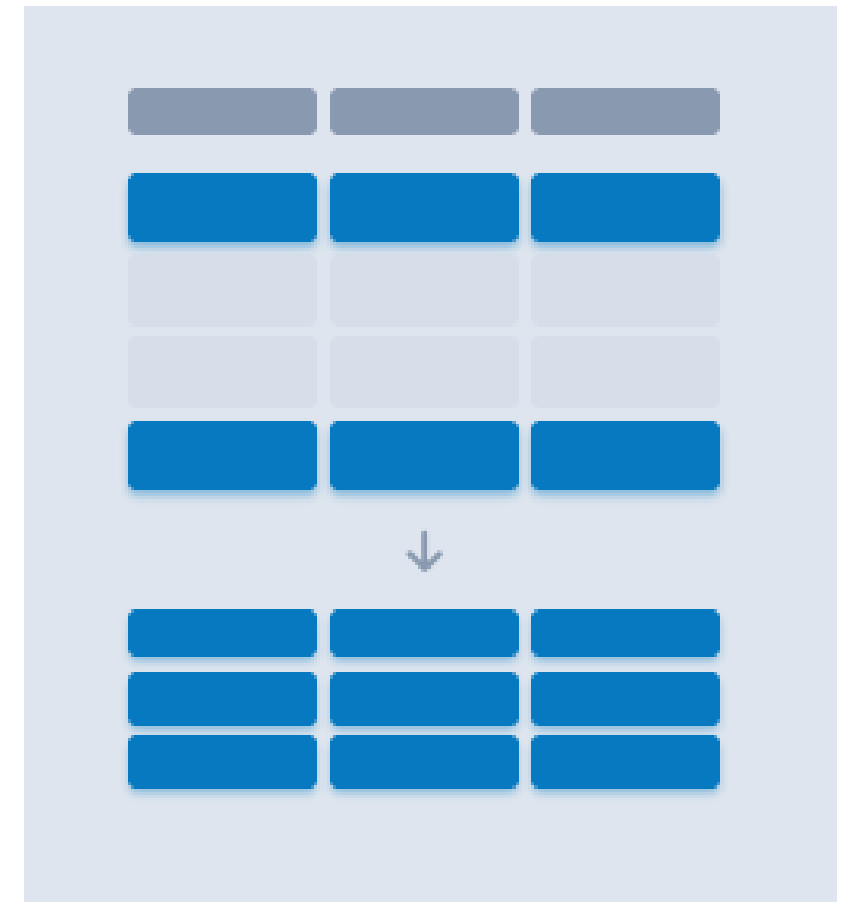
Seleccionar los datos de vendedores cuyo número de ventas es superior a 120

```
# 1. DataFrame API: Usando col() para referenciar la columna
from pyspark.sql.functions import col
df_filtrado = df.filter(col("ventas") > 120)

df_filtrado.show()
```

```
# 2. Spark SQL Nativo: la cláusula WHERE clásica de SQL
df_filtrado_sql = spark.sql("""
SELECT *
FROM tabla_ventas
WHERE ventas > 120
""")

df_filtrado.show(3)
```



2. Operaciones con DataFrames en PySpark

AGRUPACIÓN DE DATOS

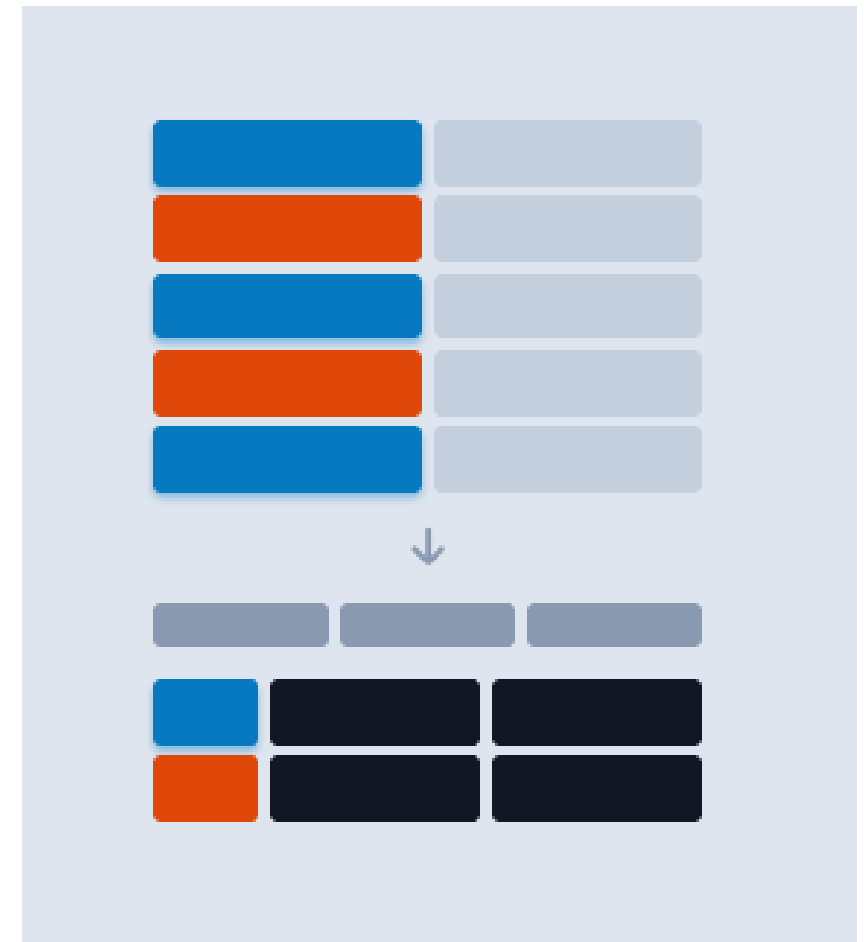
Crear grupos de datos (filas) bajo una categoría común, y aplicar sobre cada grupo una función matemática o de agregación (suma, recuento, media, etc.).

```
# 1. DataFrame API: Agrupar por ciudad y sumar la
columna ventas
df_agrupado = df.groupBy("ciudad").sum("ventas")

df_agrupado.show()
```

```
# 2. Spark SQL Nativo: Usar GROUP BY y la función de
agregación SUM()
df_agrupado_sql = spark.sql("""
SELECT ciudad, SUM(ventas) as total_ventas
FROM tabla_ventas
GROUP BY ciudad
""")

df_agrupado.show()
```



2. Operaciones con DataFrames en PySpark

CRUCE DE DATOS: JOIN

Enriquecer un DataFrame combinando datos de múltiples DataFrames mediante correspondencia entre claves de enlace (similar a los JOIN entre tablas SQL).

```
# 1. DataFrame API: cruzamos
usando la columna común "ciudad"
df_join = df.join(df_zonas,
on="ciudad", how="inner")
```

```
df_join.show()
```

```
# 2. Spark SQL Nativo: Usando INNER JOIN
estándar
```

```
df_join_sql = spark.sql("""
    SELECT v.id_usuario, v.ventas,
    v.ciudad, z.zona
    FROM tabla_ventas v
    INNER JOIN tabla_zonas z
    ON v.ciudad = z.ciudad
""")
```

```
df_join.show()
```

Datos de muestra de df:

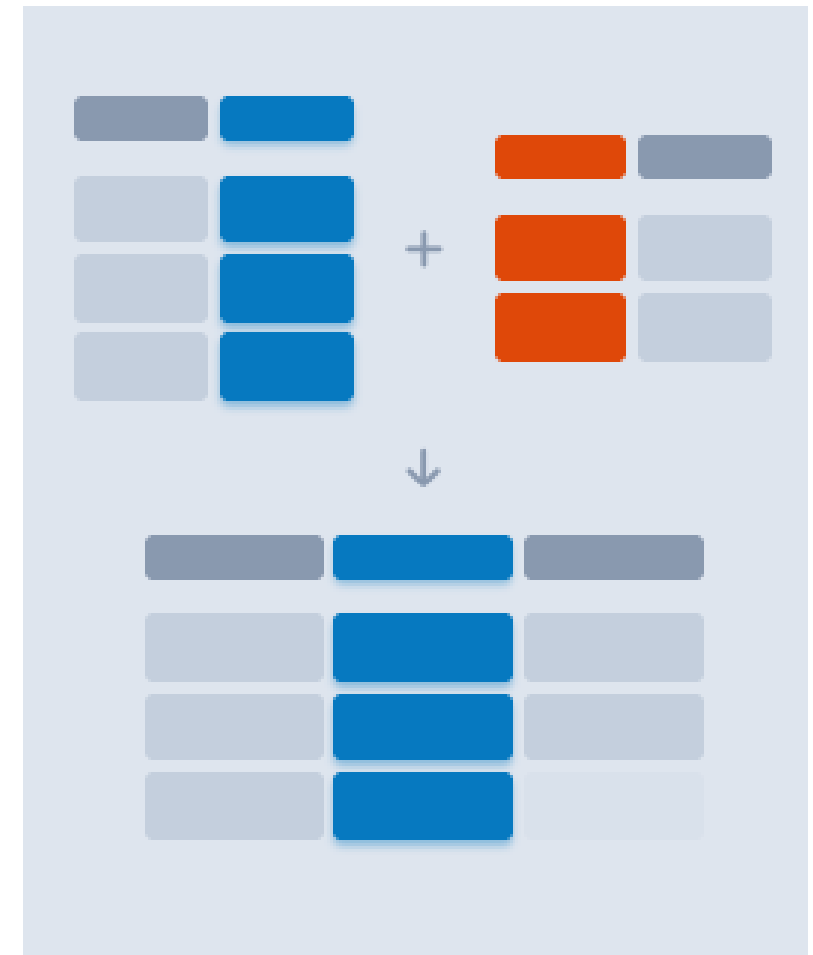
id_usuario	ventas	ciudad
1	100	Madrid
2	150	Barcelona
3	200	Valencia
4	50	Sevilla
5	120	Madrid

only showing top 5 rows

ciudad	zona
Madrid	Centro
Barcelona	Este

Modos de cruce:

inner, left,
right, outer



PySpark cuenta con varios métodos para el tratamiento de valores nulos (missing values)

- Veamos algunos de ellos

Rellenar
valores
perdidos
Con un valor
por defecto

```
# Limpieza y tratamiento de nulos sobre columnas del dataset
df_clean = df_emp.dropna(how="all") \
    .fillna({
        "income": 0.0,
        "credit_score": 300 # Puntuación crediticia por defecto
    }) \
    .withColumn("experience", coalesce(col("experience"), lit(0)))
# coalesce toma el primer valor no nulo; si 'experience' es nulo, asigna 0

df_clean.show()
```

Comenzamos eliminando filas donde todas
las columnas sean nulas

Coalesce es algo más flexible que fillna(): puedes pasarle varias columnas, y
reemplazará un valor nulo basándose primero en la existencia de valores
nulos de otras columnas, y usando el valor por defecto como último recurso

3. Uso de funciones integradas y procesamiento

Manejo de strings

Tareas como la limpieza de nombres, extracción de dominios a partir de URLs, o la unificación de formatos son tareas cotidianas al tratar con datos textuales.

- Existen funciones integradas en `pyspark.sql.functions` para el procesamiento de texto distribuido. Algunas de ellas son:

<code>concat_ws</code>	concatena múltiples columnas o expresiones mediante un separador
<code>trim</code>	elimina espacios al comienzo y final de una cadena
<code>upper</code>	convierte a mayúsculas
<code>regex_replace</code>	reemplaza patrones de texto guiándose por una expresión regular
<code>split</code>	particiona una cadena usando un delimitador

3. Uso de funciones integradas y procesamiento

Procesamiento con series temporales

También hay funciones para procesar datos de series temporales, que contienen fecha, hora, o una combinación de ambas.

De nuevo, se trata de funciones en el módulo `pyspark.sql.functions` , tales como:

<code>to_date</code>	concatena múltiples columnas o expresiones mediante un separador
<code>year, month</code>	elimina espacios al comienzo y final de una cadena
<code>date_format</code>	convierte a mayúsculas

```
+-----+-----+---+-----+
|service_date|parsed_date|year|month_name|
+-----+-----+---+-----+
| 03/01/2018| 2018-03-01|2018| March|
| 02/04/2018| 2018-02-04|2018| February|
| 03/19/2018| 2018-03-19|2018| March|
| 03/31/2018| 2018-03-31|2018| March|
| 01/01/2018| 2018-01-01|2018| January|
+-----+-----+---+-----+
```

3. Uso de funciones integradas y procesamiento

Agregaciones avanzadas

Ejemplo: encontrar los tres meses con una mayor cantidad de viajes en cada año, usando funciones de ventana.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum as spark_sum,
rank, desc

# 1. Agregación múltiple (usando la columna real
'total_rides')
df_agg = df_dates.groupBy("year", "month_name").agg(
    spark_sum("total_rides").alias("monthly_rides")
)

# 2. Window Function: Los 3 meses con más viajes de
CADA AÑO
window_spec =
Window.partitionBy("year").orderBy(desc("monthly_rides"
))

# Asignamos un ranking y filtramos el top 3
df_top3 = df_agg.withColumn("ranking",
rank().over(window_spec)) \
.filter(col("ranking") <= 3)

df_top3.show(10)
```

Uso de spark_sum(), bajo un alias, para evitar conflictos con la función nativa de Python sum()

Función de ventana: se especifica una ventana = conjunto de reglas para dividir y ordenar los datos. Las operaciones subsiguientes a partitionBy() se hacen de forma independiente para cada año

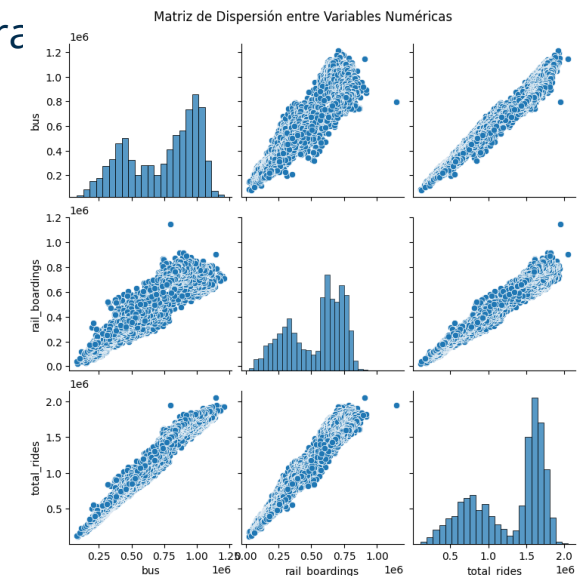
En cada partición hecha en la ventana, se asigna un rango (rank) según el valor de monthly_rides. El filtro final se queda, dentro de cada año, con los tres meses con más pasajeros

¿ Qué es el EDA?

Es un análisis preliminar de un dataset, con fines exploratorios, antes de aplicar técnicas más avanzadas como *machine learning* y uso de otros modelos de IA. Se investigan y resumen las principales características del conjunto de datos cargado en el código Python, atendiendo a:

- Inspección de los datos.
- Propiedades estadísticas básicas: media, mediana, moda, rango, desviación estándar, percentiles.
- Visualizaciones sencillas de distribuciones y variabilidad: principalmente histograma
- Distribución de los tipos de datos en los atributos.
- Detección (¡no tratamiento aún!) de valores perdidos, duplicados, outliers.

Realizar un EDA es el modo más riguroso de conocer mejor los datos con los que estás a punto de trabajar para mayores labores de análisis.



4. LAB: Limpieza y Análisis Exploratorio con consultas nativas

En la libreta Python que crearemos...

- Cargaremos un dataset de empleados creado sintéticamente a priori, para realizar un análisis matemático y visual con fines exploratorios, lo que nos llevará a detectar datos contradictorios y de baja calidad en el escenario mostrado.

EJERCICIO PROPUESTO: realizar un EDA sobre dos datasets de tu elección, que incluya dos visualizaciones similares a las realizadas.

